SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – Experiments

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO3 – Precision	**Assessment:**	Assessment Tool 1 – Experiments
Weightage:	50% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	Madhavan. S	**Reg. No.:**	192425113
Section / Batch:	CSE-AI/2024/D	**Date:**	06/08/2026

Code of Conduct

*I **Madhavan. S (192425113)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*
Signature of the Student: **Madhavan. S**

Instructions

- Perform all experiments using Google Colab.
- Create a separate notebook (.ipynb) for the assessment.
- Ensure all code is executable without errors.
- Include appropriate comments explaining the logic used.
- Complete all three experiments in a single Google Colab notebook.
- Execute all cells and display the outputs for the given test cases.
- Save the notebook with the naming convention: RegNo_Name_CO3-AT1.ipynb
- Upload the notebook to your GitHub repository.
- Ensure the repository is public or accessible to the instructor.
- Submit the following:
- GitHub Repository Link in GCR
- Google Colab Share Link in GCR
- Screenshots of uploaded coding and output files as printout.
- Duration: 30 minutes.

Section / Topic	Focus	Experiment Questions
Context-Free Grammars for English Syntax, Context-Free Rules and Trees, Sentence-Level Constructions, Parsing, Top-Down Parsing, Earley Parsing	Design a Python program to validate sentences using CFG production rules and construct parse tree representations for valid sentences.	Q1
Agreement, Feature Structures, Sub Categorization	Design a Python program to verify subject–verb agreement and validate grammatical constraints using feature structures and subcategorization rules.	Q2
Probabilistic Context-Free Grammars (PCFGs)	Design a Python program to parse sentences and compute sentence probabilities using probabilistic grammar production rules.	Q3

Experiment 1: Context-Free Grammar Rule Validation and Parse Tree Construction

Experiment Description

Design a Python program to validate whether a sentence follows the given Context-Free Grammar (CFG) rules and generate its parse tree representation.

Grammar:

```
S  → NP VP
NP → Det N
VP → V NP

Det → the | a
N   → student | teacher | book
V   → reads | likes
```

Task

1. Accept a sentence as input.
2. Verify whether it conforms to the CFG.
3. Construct and return the parse tree.
4. Return "Invalid Sentence" if parsing fails.

Function Prototype

```
def parse_cfg(sentence):
        pass
```

Test Cases

Input	Expected Output
the student reads a book	Valid Parse Tree
a teacher likes the book	Valid Parse Tree
student reads book	Invalid Sentence
the book likes a teacher	Valid Parse Tree
reads the student book	Invalid Sentence

Experiment 2: Agreement and Feature Structure Checking

Experiment Description

Design a Python program that verifies subject–verb agreement using feature structures containing Number and Person attributes.

Feature Rules

```
he     → singular, third person
she    → singular, third person
it     → singular, third person
they   → plural

runs   → singular verb
writes → singular verb

run    → plural verb
write  → plural verb
```

Task

1. Accept Subject and Verb.
2. Compare feature structures.
3. Return True if agreement holds.
4. Return False otherwise.

Function Prototype

```
def check_agreement(subject, verb):
    pass
```

Test Cases

Subject	Verb	Expected
he	runs	True

Subject	Verb	Expected
he	run	False
they	run	True
they	writes	False
she	writes	True

Experiment 3: Probabilistic Context-Free Grammar (PCFG) Parsing

Experiment Description

Implement a Python program to compute the probability of a sentence using the given PCFG.

Grammar

```
S  → NP VP       [1.0]

NP → John        [0.6]
NP → Mary        [0.4]

VP → runs        [0.5]
VP → walks       [0.5]
```

Probability Formula

```
P(Sentence)
=
P(S→NP VP)
× P(NP)
× P(VP)
```

Task

1. Accept a two-word sentence.
2. Parse using the PCFG.
3. Compute sentence probability.
4. Return 0 if sentence cannot be generated.

Function Prototype

```
def sentence_probability(sentence):
    pass
```

Test Cases

Input Sentence	Expected Probability
John runs	0.30
John walks	0.30
Mary runs	0.20
Mary walks	0.20
Peter runs	0.00

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts

Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Q. No. / Criteria Level	Understanding of Concepts	Experimental Design	Use of Tools	Analysis of Results	Documentation	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Experiment 1: Context-Free Grammar Rule Validation and Parse Tree Construction

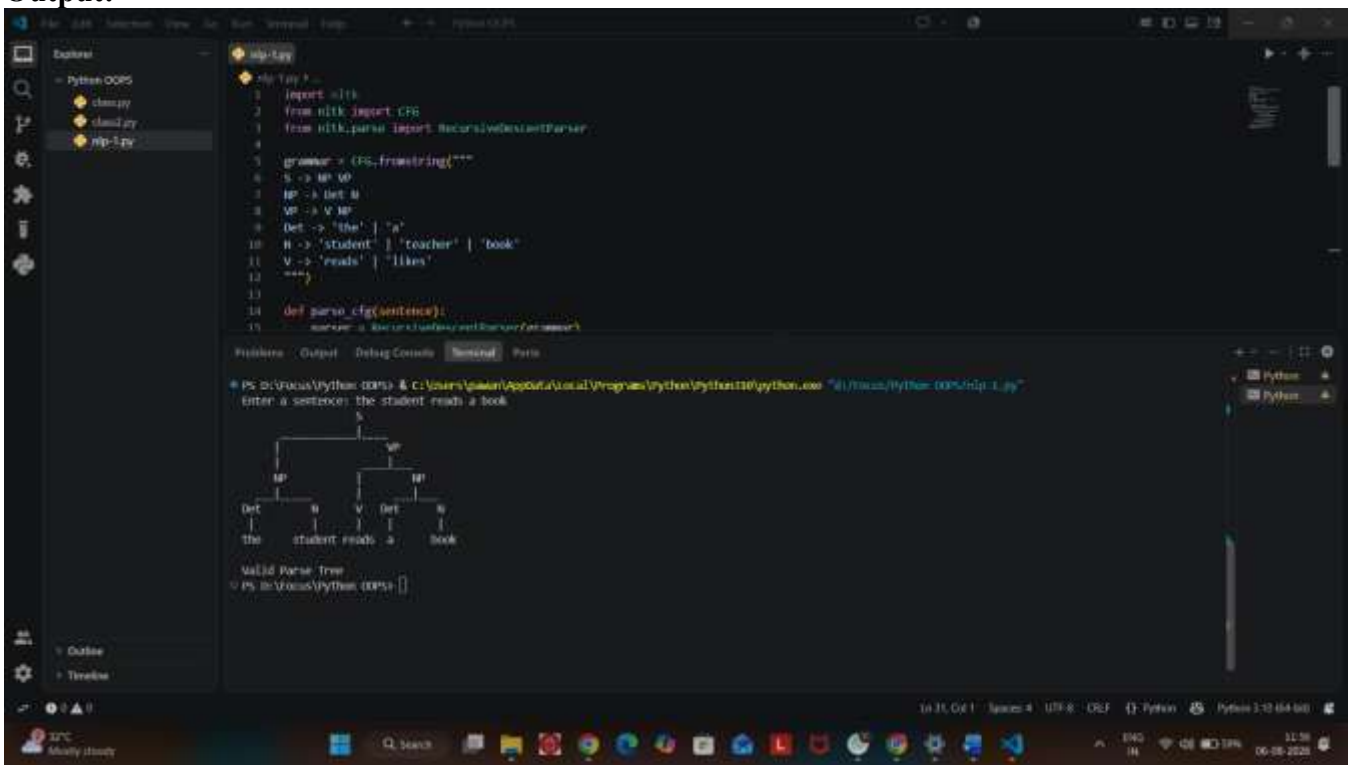
Aim:
To design a Python program to validate whether a sentence follows given Context-Free Grammar (CFG) rules and construct its parse tree representation.

Algorithm:
1. Start
2. Define the Context-Free Grammar production rules for S, NP, VP, Det, N and V
3. Read the input sentence and split it into individual words
4. Check whether each word belongs to the correct grammar category
5. Verify whether the sentence structure matches S -> NP VP, NP -> Det N, VP -> V NP
6. If the sentence matches the grammar structure, construct the parse tree
7. If the sentence does not match the grammar, return "Invalid Sentence"
8. Display the parse tree or the invalid message
9. Stop

Python Code:
```
import nltk
from nltk import CFG
from nltk.parse import RecursiveDescentParser

grammar = CFG.fromstring("""
S -> NP VP
NP -> Det N
VP -> V NP
Det -> 'the' | 'a'
N -> 'student' | 'teacher' | 'book'
V -> 'reads' | 'likes'
""")

def parse_cfg(sentence):
    parser = RecursiveDescentParser(grammar)
    words = sentence.lower().split()
    try:
        trees = list(parser.parse(words))
        if trees:
            for tree in trees:
                tree.pretty_print()
            return "Valid Parse Tree"
        else:
            return "Invalid Sentence"
    except ValueError:
        return "Invalid Sentence"

sentence = input("Enter a sentence: ")
result = parse_cfg(sentence)
print(result)
```

Sample Input:
Enter a sentence: the student reads a book

Output:



Result:

The Python program to validate sentences using CFG production rules and construct their parse tree representation was executed successfully, and the output was verified for all the given test cases.

Experiment 2: Agreement and Feature Structure Checking

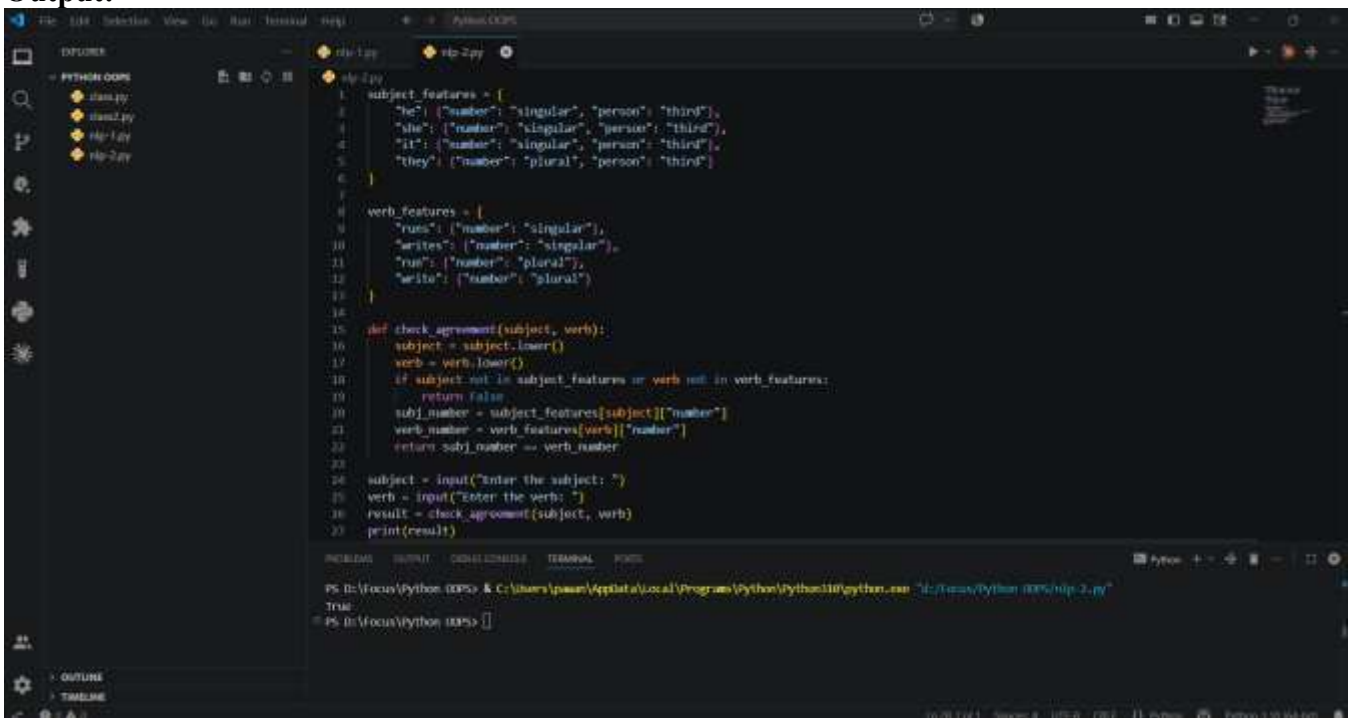
Aim:
To design a Python program that verifies subject-verb agreement using feature structures containing Number and Person attributes.

Algorithm:
1. Start
2. Define feature structures for subjects with Number and Person attributes
3. Define feature structures for verbs with Number attribute
4. Read the subject and verb as input
5. Retrieve the feature structure of the given subject and verb
6. Compare the Number feature of the subject with the Number feature of the verb
7. If the features match, return True
8. If the features do not match, return False
9. Display the result
10. Stop

Python Code:
```
subject_features = {
    "he": {"number": "singular", "person": "third"},
    "she": {"number": "singular", "person": "third"},
    "it": {"number": "singular", "person": "third"},
    "they": {"number": "plural", "person": "third"}
}

verb_features = {
    "runs": {"number": "singular"},
    "writes": {"number": "singular"},
    "run": {"number": "plural"},
    "write": {"number": "plural"}
}

def check_agreement(subject, verb):
    subject = subject.lower()
    verb = verb.lower()
    if subject not in subject_features or verb not in verb_features:
        return False
    subj_number = subject_features[subject]["number"]
    verb_number = verb_features[verb]["number"]
    return subj_number == verb_number

subject = input("Enter the subject: ")
verb = input("Enter the verb: ")
result = check_agreement(subject, verb)
print(result)
```

Sample Input:
Enter the subject: he
Enter the verb: runs

Output:



Result:

The Python program to verify subject-verb agreement using feature structures was executed successfully, and the output was verified for all the given test cases.

Experiment 3: Probabilistic Context-Free Grammar (PCFG) Parsing

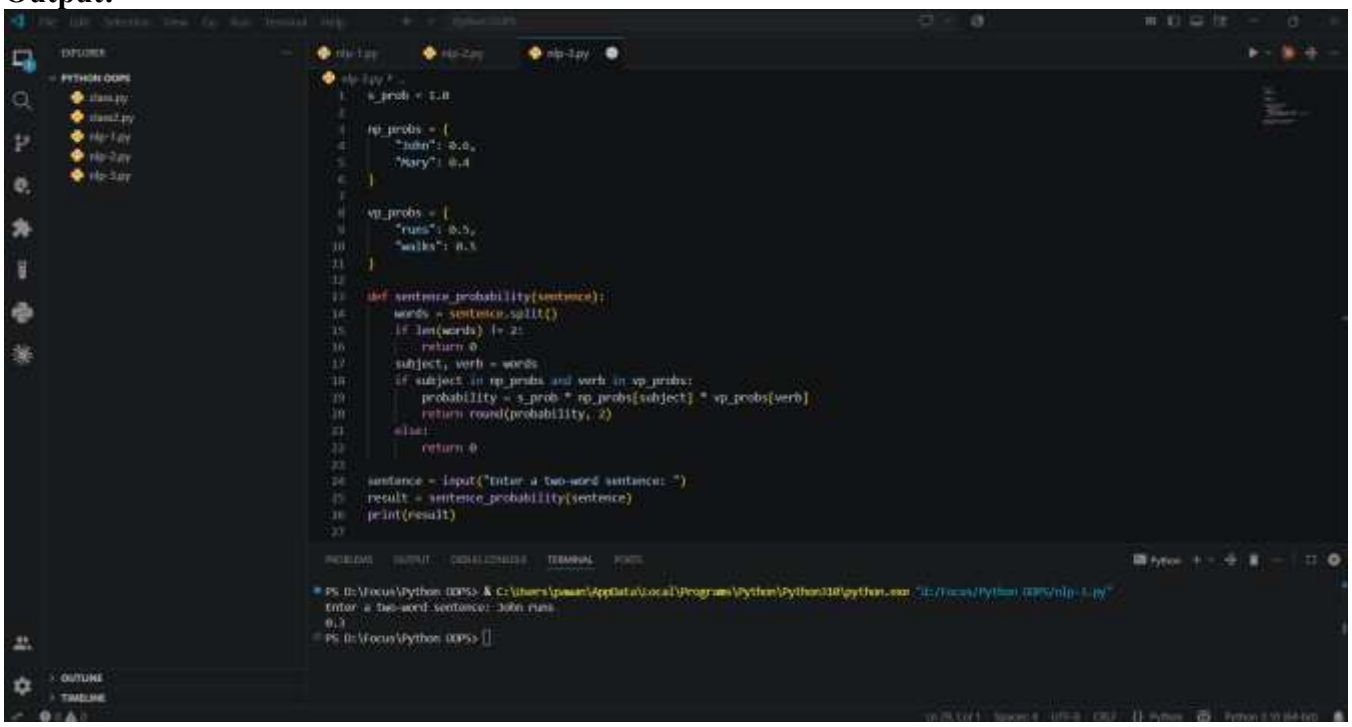
Aim:
To implement a Python program to compute the probability of a sentence using a given Probabilistic Context-Free Grammar (PCFG).

Algorithm:
1. Start
2. Define the PCFG production rules along with their associated probabilities
3. Read a two-word sentence as input
4. Split the sentence into subject and verb
5. Check whether the subject exists in the NP production rules
6. Check whether the verb exists in the VP production rules
7. If both exist, compute the sentence probability using P(S) x P(NP) x P(VP)
8. If either word does not exist in the grammar, return 0
9. Display the computed probability
10. Stop

Python Code:
```
s_prob = 1.0
np_probs = {
    "John": 0.6,
    "Mary": 0.4
}
vp_probs = {
    "runs": 0.5,
    "walks": 0.5
}
def sentence_probability(sentence):
    words = sentence.split()
    if len(words) != 2:
        return 0
    subject, verb = words
    if subject in np_probs and verb in vp_probs:
        probability = s_prob * np_probs[subject] * vp_probs[verb]
        return round(probability, 2)
    else:
        return 0
sentence = input("Enter a two-word sentence: ")
result = sentence_probability(sentence)
print(result)
```

Sample Input:
Enter a two-word sentence: John runs

Output:



Result:

The Python program to compute sentence probability using a Probabilistic Context-Free Grammar was executed successfully, and the output was verified for all the given test cases.